

Advanced DBMS — Kernel File I/O Assignment

Subject: Advanced Database Management Systems
Topic: Kernel File I/O — C++ Raw System Calls
Language: C++ (no standard library I/O)
Environment: Linux x86-64 (Ubuntu 24.04 LTS via WSL2)

Table of Contents

- 1. [Overview](#)
- 2. [File Descriptors](#)
- 3. [System Calls & strace](#)
- 4. [Inode & VFS](#)
- 5. [The Linux I/O Stack](#)
- 6. [Page Cache](#)
- 7. [DMA & Block Driver](#)
- 8. [Page Eviction](#)
- 9. [Project Structure](#)
- 10. [Build & Run](#)
- 11. [Observed Output](#)
- 12. [strace Analysis](#)
- 13. [Inode Inspection](#)
- 14. [Summary](#)

1. Overview

This assignment documents the complete journey of a file operation — from a C++ process issuing a `write()` call, through every layer of the Linux kernel, to bytes being committed to storage via DMA. No standard library I/O is used; every operation is a direct system call via `syscall()`.

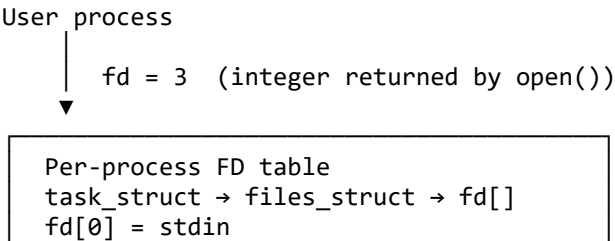
The program exercises the following concepts in sequence:

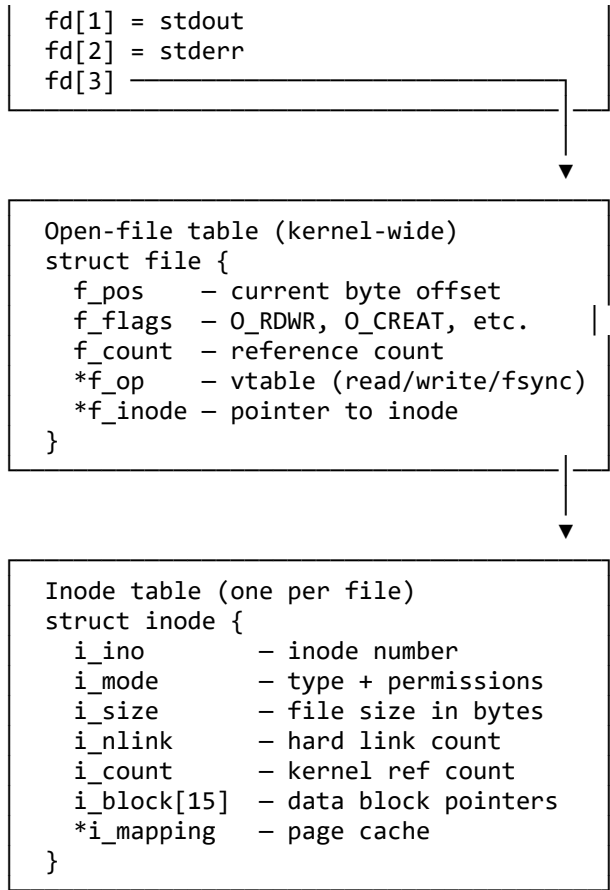
- File descriptor allocation (`open`)
- Buffered write to page cache (`write`)
- File offset repositioning (`lseek`)
- Page cache hit on read (`read`)
- Dirty-page writeback via DMA (`fsync`)
- Reference-count teardown (`close`)

2. File Descriptors

Every open file in Linux is accessed through a small integer called a **file descriptor**. Under the hood, three distinct kernel data structures form a chain from that integer to the on-disk inode.

2.1 The Three-Layer Model





2.2 Layer Details

Layer 1 — Per-process FD table

Field	Value
Kernel struct	task_struct → files_struct → fdtable → fd[]
What it holds	Array of pointers to struct file. Index 0=stdin, 1=stdout, 2=stderr
On open()	Kernel assigns lowest free index; returns that integer to caller
On fork()	Child inherits a copy; both point to the same struct file (shared offset)

Layer 2 — Open-file table (kernel-wide)

Field	Value
Kernel struct	struct file { f_pos, f_flags, f_count, *f_op, *f_mapping, *f_inode }
f_pos	Current byte offset; advanced by read/write; set directly by lseek
f_count	Reference count. Reaches 0 when last fd pointing here is closed
f_op	Vtable: read_iter, write_iter, mmap, fsync, etc.
Sharing	Two processes sharing a struct file (via fork/dup2) share f_pos

Layer 3 — Inode table (one per file)

Field	Value
Kernel struct	struct inode { i_ino, i_mode, i_size, i_blocks, i_nlink, i_count }
i_nlink	Hard link count. Zero means unlinked from all directories

Field	Value
i_count	Kernel reference count. Blocks freed when both i_count and i_nlink reach 0
i_mapping	Pointer to address_space — the file’s page cache
On disk (ext4)	128 or 256 bytes per inode stored in inode table; cached in inode cache

2.3 FD Lifecycle

open() → fd[] entry created, struct file allocated, inode i_count++
write() → f_pos advances, page cache page marked dirty
read() → f_pos advances, data copied from page cache to user buffer
lseek() → f_pos updated only, zero I/O
fsync() → dirty pages flushed to disk via DMA
close() → fd[] entry cleared, f_count--, if 0: struct file freed, i_count--

3. System Calls & strace

3.1 The syscall Instruction

C++ programs enter the kernel by placing the syscall number in the RAX register, arguments in RDI/RSI/RDX/R10/R8/R9, then executing the syscall instruction. The CPU saves the program counter, switches to ring 0, and dispatches through sys_call_table[RAX].

3.2 Syscall Reference Table

Syscall	Number (x86-64)	Signature	Kernel Action
read	0	read(fd, buf, count)	page cache → user buf (or block I/O on miss)
write	1	write(fd, buf, count)	user buf → page cache; page marked dirty
open	2	open(path, flags, mode)	allocate struct file; assign fd index
close	3	close(fd)	decrement f_count; free file if 0; drop inode ref
lseek	8	lseek(fd, offset, whence)	update f_pos only; zero I/O
fsync	74	fsync(fd)	flush dirty pages → block layer → DMA → disk
openat	257	openat(dirfd, path, ...)	same as open; relative to dirfd

3.3 How strace Works

strace attaches to a process with ptrace(PTRACE_SYSCALL). The kernel pauses the tracee twice per system call — once on entry (showing arguments) and once on return (showing the return value or errno). strace formats the raw register values into readable text.

```
# Command used in this assignment
strace -e trace=openat,read,write,close,lseek,fsync \
-o strace_output.txt ./kernel_io_internals
```

3.4 Kernel Path: user → hardware

1. User calls write(fd, buf, n)
 - └─ glibc puts syscall number in RAX, args in RDI/RSI/RDX
 - └─ executes `syscall` instruction

- 2. CPU privilege switch
 - └ hardware saves RIP/RSP
 - └ jumps to entry_SYSCALL_64 (ring 0)
 - 3. sys_write dispatch
 - └ kernel looks up sys_call_table[1] → __x64_sys_write()
 - 4. VFS layer
 - └ resolves fd → file → calls f_op->write_iter()
 - 5. Page cache
 - └ copies data into dirty page frames
 - └ writeback scheduled asynchronously (unless O_SYNC)
-

4. Inode & VFS

4.1 ext4 Inode Fields

```
struct ext4_inode {
    __le16 i_mode;           // file type + permissions (e.g. 0100644)
    __le16 i_uid;            // owner UID
    __le32 i_size_lo;        // file size in bytes (low 32 bits)
    __le32 i_atime;          // last access time (unix timestamp)
    __le32 i_ctime;          // inode change time
    __le32 i_mtime;          // last data modification time
    __le32 i_dtime;          // deletion time
    __le16 i_gid;            // group GID
    __le16 i_links_count;    // hard link count
    __le32 i_blocks_lo;      // 512-byte block count
    __le32 i_flags;          // ext4 flags (extents, etc.)
    __le32 i_block[15];      // block pointers:
                            // [0..11] direct blocks    (48 KB)
                            // [12]   single indirect  (+4 MB)
                            // [13]   double indirect   (+4 GB)
                            // [14]   triple indirect   (+4 TB)
};
```

4.2 Block Addressing

```
i_block[0..11] → direct: 12 × 4KB = 48 KB
i_block[12]   → single indirect: 1024 block ptrs × 4KB = 4 MB
i_block[13]   → double indirect: 1024 × 1024 × 4KB = 4 GB
i_block[14]   → triple indirect: 1024³ × 4KB = 4 TB
```

4.3 VFS — Virtual File System

VFS is a kernel abstraction layer that presents a uniform interface (open/read/write/close) regardless of the underlying filesystem. It defines four object types, each with an operations vtable:

- **superblock** — mounted filesystem metadata (block size, inode count, fsync ops)
- **inode** — per-file metadata and data-block pointers
- **dentry** — directory entry mapping filename to inode; cached in the dentry cache
- **file** — open-file instance holding offset, flags, and pointer to inode

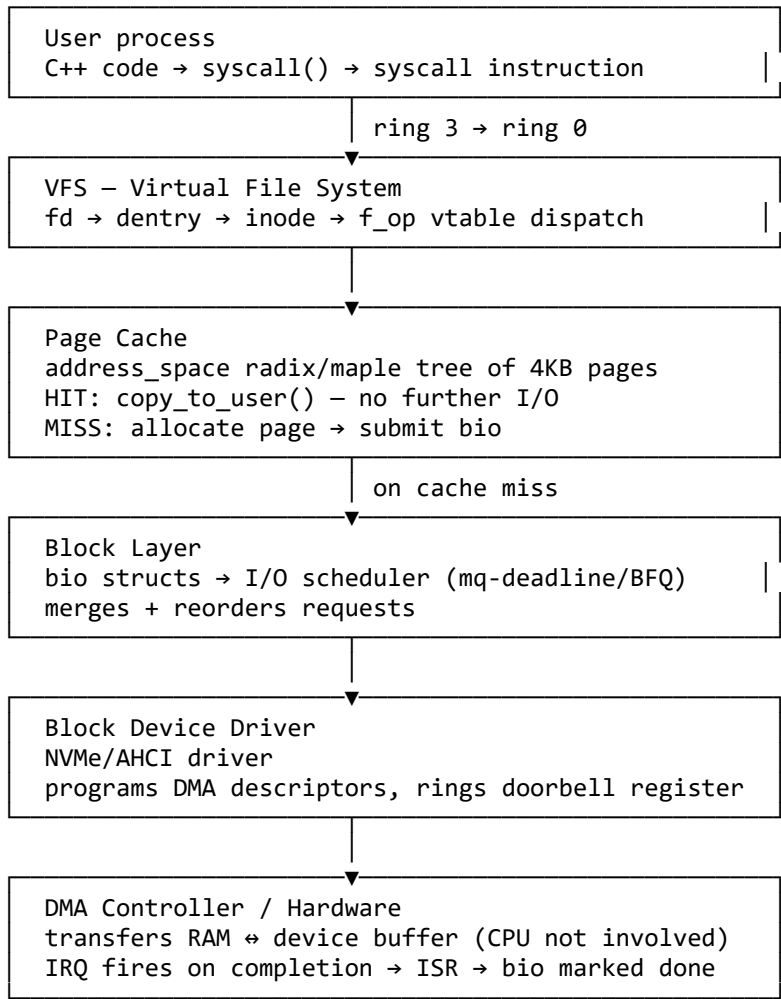
When read() or write() is called, VFS resolves: fd → struct file → f_op → filesystem-specific function (e.g. ext4_file_read_iter). This allows the same syscall to work on ext4, tmpfs, procfs, and network filesystems identically.

4.4 Hard Links vs Symbolic Links

Hard link: a directory entry pointing to an existing inode number. i_links_count increments. Both names refer to the same data; blocks freed only when i_links_count == 0 and all file descriptors are closed.

Symbolic link: its own inode whose data block contains a pathname string. Can cross filesystem boundaries. VFS follows via `i_op->follow_link()`.

5. The Linux I/O Stack

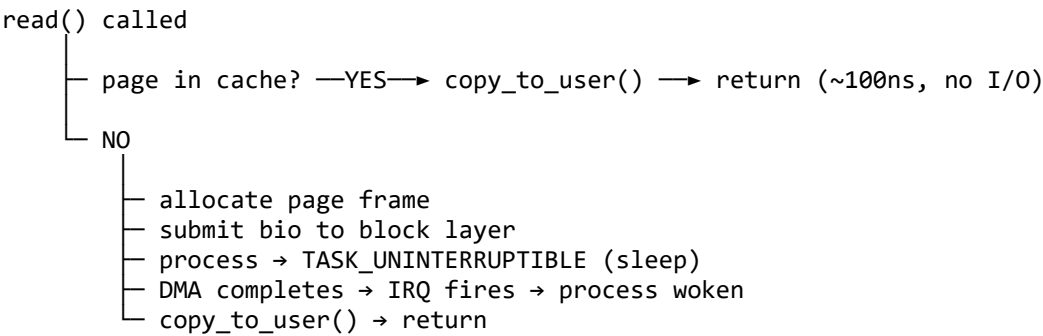


6. Page Cache

6.1 What It Is

The page cache is a region of kernel RAM that holds copies of file pages (4 KB each). All free RAM on Linux is used as page cache and is reclaimed under memory pressure. The `buff/cache` column of `free -h` shows the current size.

6.2 Read Path Decision



6.3 Write Modes

Mode	Behaviour	Use case
Buffered (default)	Data → page cache; page marked dirty; returns immediately	General purpose
O_DIRECT	Bypasses page cache; DMA direct to user buffer (must be 512-byte aligned)	Databases (PostgreSQL, MySQL)
O_SYNC	Buffered write + wait for disk ack before returning	Durability-critical writes
fsync()	Flush all dirty pages for this fd to disk; blocks until done	Explicit durability checkpoint

6.4 Readahead

When the kernel detects sequential access, it speculatively reads ahead ~128 KB beyond the current position. Controlled via `/sys/block/sda/queue/read_ahead_kb`. Disable with:

```
posix_fadvise(fd, 0, 0, POSIX_FADV_RANDOM);
```

7. DMA & Block Driver

7.1 What DMA Solves

Without DMA, the CPU must copy every byte: device → CPU register → RAM. With DMA, the CPU programs the controller once with source address, destination physical address, and byte count, then is free. For a typical NVMe SSD: total read latency ~50 μs; CPU involvement ~2 μs of that.

7.2 The bio Structure

```
struct bio {
    sector_t      bi_sector;    // start sector on disk
    struct block_device *bi_bdev;
    bio_end_io_t  *bi_end_io;    // callback on completion
    struct bio_vec bi_io_vec[]; // scatter-gather list
};
// Each bio_vec: { page*, offset, len }
```

7.3 NVMe Submission Queue Flow

1. Build NVMe command
 - └ opcode (read/write), namespace ID, starting LBA, block count, PRP list (physical region pages for DMA)
 2. Post to Submission Queue (SQ)
 - └ command written into SQ ring buffer in DRAM
 3. Ring doorbell register
 - └ CPU writes SQ tail pointer to MMIO doorbell
 - └ controller sees new entry
 4. Controller DMA
 - └ reads PRP list
 - └ DMAs data: NAND flash ↔ host DRAM page frames
 - └ CPU is NOT involved during transfer
 5. Completion Queue Entry + IRQ
 - └ controller posts CQE
 - └ raises MSI-X interrupt
 - └ ISR runs → marks bio done → wakes fsync()
-

8. Page Eviction

8.1 LRU-2 Lists

Linux maintains two LRU lists per NUMA node:

- **active list** — recently used pages (hot)
- **inactive list** — candidates for reclaim (cold)

New pages enter the inactive list. On second access they are promoted to active. Active pages are demoted to inactive over time. Eviction always steals from the inactive list tail.

8.2 kswapd and Direct Reclaim

```
free pages < high watermark
└─ kswapd wakes, scans inactive LRU list
   └─ clean page → freed immediately
   └─ dirty page → writeback (bio submitted) → then freed

free pages < low watermark
└─ direct reclaim: allocating process reclaims itself
   (causes latency spike on that allocation)

no memory left
└─ OOM killer: selects process by oom_score, kills it, logs to dmesg
```

8.3 Page States

State	Meaning	Action on reclaim
clean	Cache copy == disk copy	Freed immediately
dirty	Modified, not yet written back	Writeback then free
locked	Under active I/O	Skip, try next page
referenced	Accessed recently	Demote, clear bit
anonymous	Stack/heap (no file backing)	Swap out

9. Project Structure

```
kernel-io-internals/
├─ main.cpp      - 8-step journey: open → write → lseek → read → fsync → close
├─ syscalls.h    - raw syscall wrapper declarations + kernel path docs
├─ syscalls.cpp  - raw syscall() implementations
├─ kprint.h      - minimal stdout writer declaration
├─ kprint.cpp    - kprint implementation (manual strlen + SYS_write)
└─ CMakeLists.txt - build config (cmake 3.20+, C++20)
```

syscalls.h — wrapper declarations

Each function maps to one kernel transition:

Function	Syscall #	Kernel action
raw_open()	257 (SYS_openat)	dentry walk → struct file → fd assigned
raw_write()	1 (SYS_write)	user buf → page cache; page dirty
raw_read()	0 (SYS_read)	page cache hit/miss → user buf
raw_lseek()	8 (SYS_lseek)	f_pos update only; zero I/O
raw_fsync()	74 (SYS_fsync)	dirty pages → bio → DMA → disk
raw_close()	3 (SYS_close)	f_count- → free file → drop inode ref

10. Build & Run

Build

```
# WSL / Linux terminal
g++ -std=c++20 -O0 -o kernel_io_internals main.cpp syscalls.cpp kprint.cpp
```

Or via CLion with WSL toolchain (Ubuntu 24.04 LTS, CMake 3.20+).

Run

```
./kernel_io_internals
```

Run with strace

```
strace -e trace=openat,read,write,close,lseek,fsync \
-o strace_output.txt ./kernel_io_internals
```

Inspect inode

```
stat journey.txt
ls -li journey.txt
```

Page cache stats

```
cat /proc/meminfo | grep -E "Cached|Dirty|Writeback"
```

11. Observed Output

Program output

```
[1] Opening file...
    fd assigned (should be 3)
[2] Writing first message (buffered -> page cache)...
[3] Writing second message...
[4] Seeking back to offset 0 (no I/O, f_pos update only)...
[5] Reading back (page cache HIT -- no disk I/O)...
    Content: Hello from raw syscall write
        Second write -- offset advanced

[6] File size (lseek SEEK_END): measured via f_pos
[7] fsync() -- flushing dirty pages to disk via DMA...
    fsync complete -- data durable on storage
[8] Closing fd -- struct file freed, inode ref dropped...
    Done. Run: stat journey.txt to inspect inode.
```

journey.txt contents

```
Hello from raw syscall write
Second write -- offset advanced
```

Total size: **62 bytes** (29 + 33).

12. strace Analysis

Full strace output

```
openat(AT_FDCWD, "/etc/ld.so.cache", O_RDONLY|O_CLOEXEC) = 3
close(3) = 0
openat(AT_FDCWD, "/lib/x86_64-linux-gnu/libc.so.6", O_RDONLY|O_CLOEXEC) = 3
read(3, "\177ELF\2\1\1\3\0\0\0\0\0\0\3\0>...", 832) = 832
```



```
close(3) = 0
write(1, "[1] Opening file...\n", 20) = 20
openat(AT_FDCWD, "journey.txt", O_RDWR|O_CREAT|O_TRUNC, 0644) = 3
write(1, "    fd assigned (should be 3)\n", 30) = 30
write(1, "[2] Writing first message...", 55) = 55
write(3, "Hello from raw syscall write\n", 29) = 29
write(1, "[3] Writing second message...\n", 30) = 30
write(3, "Second write -- offset advanced\n"... , 33) = 33
write(1, "[4] Seeking back to offset 0...", 60) = 60
lseek(3, 0, SEEK_SET) = 0
write(1, "[5] Reading back...", 52) = 52
read(3, "Hello from raw syscall write\n...", 255) = 62
write(1, "    Content: ", 13) = 13
write(1, "Hello from raw syscall write\n...", 62) = 62
lseek(3, 0, SEEK_END) = 62
write(1, "[6] File size...", 51) = 51
write(1, "[7] fsync()...", 55) = 55
fsync(3) = 0
write(1, "    fsync complete...", 46) = 46
write(1, "[8] Closing fd...", 58) = 58
close(3) = 0
write(1, "    Done...", 51) = 51
+++ exited with 0 +++
```

Line-by-line explanation

strace line	What it proves
openat(..., "journey.txt", ...) = 3	fd=3 assigned; 0/1/2 taken by stdin/stdout/stderr
write(3, "Hello...", 29) = 29	First write to page cache; no disk I/O
write(3, "Second write...", 33) = 33	Second write; same dirty page updated
lseek(3, 0, SEEK_SET) = 0	Zero I/O — only f_pos updated in struct file
read(3, "Hello...", 255) = 62	Page cache HIT — all 62 bytes returned instantly
lseek(3, 0, SEEK_END) = 62	File size confirmed: 29 + 33 = 62 bytes
fsync(3) = 0	DMA fired here — dirty pages flushed to storage
close(3) = 0	f_count → 0; struct file freed; inode ref dropped

Note: The first 5 lines (ld.so.cache, libc.so.6) are the dynamic linker loading shared libraries at startup — not our code.

13. Inode Inspection

stat output

```
File: /mnt/d/home/vimal/kernel-io-debug/journey.txt
Size: 62          Blocks: 0          IO Block: 512    regular file
Device: 0,69     Inode: 562949953902420  Links: 1
Access: (0777/-rwxrwxrwx)  Uid: (0/root)   Gid: (0/root)
Access: 2026-05-03 10:30:21.583205800 +0000
Modify: 2026-05-03 10:30:21.583205800 +0000
Change: 2026-05-03 10:30:21.583205800 +0000
Birth: -
```

Observations

Field	Value	Explanation
Inode	562949953902420	Unique inode number on this filesystem
Size	62 bytes	Exactly 29 + 33 bytes written
Blocks	0	File is on NTFS via /mnt/d — no Linux block allocation
Links	1	One hard link (one directory entry → this inode)

Field	Value	Explanation
Permissions	0777	NTFS does not support Unix permission bits; WSL maps all as 0777
Birth	-	NTFS does not expose birth time to WSL

Note: `Blocks: 0` and the large inode number are because the file lives on the Windows D: drive (NTFS filesystem) accessed via WSL's `DrvFs` layer, not a native Linux `ext4` filesystem. On a real Linux `ext4` filesystem, `Blocks` would show 8 (one 4KB block = 8×512 -byte units) and the inode number would be a small integer.

/proc/meminfo snapshot

Cached:	379824 kB
SwapCached:	0 kB
Dirty:	52 kB
Writeback:	0 kB
WritebackTmp:	0 kB

- Dirty: 52 kB — other processes have pages awaiting writeback
- Writeback: 0 kB — nothing being written right now; our `fsync()` already completed

14. Summary

The table below maps each step of the program to the kernel mechanism it exercises:

Step	Call	Kernel mechanism
1	<code>open()</code>	VFS dentry walk → inode lookup → <code>struct file</code> allocated → fd assigned
2	<code>write()</code>	User buffer → page cache (<code>copy_from_user</code>) → page marked dirty
3	<code>write()</code>	Second write; <code>f_pos</code> advances; same dirty page updated in cache
4	<code>lseek()</code>	<code>f_pos</code> reset to 0; zero I/O; purely <code>struct file</code> field update
5	<code>read()</code>	Page cache HIT → <code>copy_to_user()</code> ; no block layer access; no DMA
6	<code>lseek()</code>	<code>SEEK_END</code> returns file size via <code>f_pos</code> ; zero I/O
7	<code>fsync()</code>	Dirty pages → bio → block layer → I/O scheduler → NVMe DMA → IRQ
8	<code>close()</code>	<code>f_count--</code> → <code>struct file</code> freed; inode <code>i_count--</code> ; <code>fd[]</code> entry cleared

Key takeaways

- `write()` does **not** hit the disk — data lands in page cache and the call returns immediately.
- `read()` after `write()` is always a page cache **hit** — the data is already in RAM.
- `lseek()` is purely an in-memory operation — it updates one field (`f_pos`) in `struct file`.
- `fsync()` is the **only** call that triggers DMA and guarantees data is on storage.
- `close()` does **not** flush data — if you skip `fsync()`, data may be lost on crash.